

Lab 4 — Vibe Coding Chat History

Project: Maze Runner (Pygame)
Repository: muaz-q/12_maze_runner
Assignment: Lab 4 — VibeCoding

Purpose: This document records the Vibe Coding workflow used to inspect the starter repository, fix the existing broken functionality, and implement the four README tasks individually. Each feature was handled as a separate task and verified before moving to the next.

1. Initial repository analysis

User/Vibe Coding Prompt

Read the README.md and inspect the entire Maze Runner Python/Pygame codebase.

Do NOT modify any files yet.

Understand:

1. How the maze is generated.
2. How the player moves.
3. How the start and exit positions are handled.
4. How collision with walls works.
5. How the maze and player are rendered.
6. How keyboard input is handled.
7. How the R key regenerates the maze.
8. Any existing bugs or broken functionality.

Compare the implementation against the README requirements.

Identify any existing broken functionality or issues in the starter code.

Also identify which files will need to be modified for each of the four tasks:

- Task 1: Shortest Path Hint
- Task 2: Fog of War
- Task 3: Timer Leaderboard
- Task 4: Difficulty Tiers

Do not implement any of the four tasks.

Result / AI Work Summary

The code review identified one clear starter-code defect: wall collision was not actually checking the maze's N/S/E/W wall data. The player could therefore pass through interior walls. The review also mapped the four README tasks to the relevant game files.

2. Fix existing broken functionality

User/Vibe Coding Prompt

Fix ONLY the existing wall-collision bug identified during the code review.

Modify game/player.py so the player cannot move through any maze wall.

Requirements:

- Use the existing walls structure from game/maze.py.
- walls are stored as [N, S, E, W].
- Preserve W/A/D/S and arrow-key controls and movement speed.
- Preserve maze generation and R.
- Do not implement Tasks 1–4.
- Keep the existing architecture.

Test movement against walls and verify the player cannot escape or pass through interior walls.

Result / AI Work Summary

The wall-collision logic was corrected in game/player.py to inspect the maze wall data as well as maze boundaries. The change was tested locally and committed separately as: Fix broken maze wall collision.

3. Task 1 — Shortest Path Hint

User/Vibe Coding Prompt

Implement ONLY Task 1 from README.md: Shortest Path Hint.

Implement a BFS solver for the maze. Press H to toggle the solution path. The path must respect actual maze walls, update after R, and preserve existing movement and wall collision. Do not implement Tasks 2–4.

After implementation, verify H shows/hides the path, R produces a newly calculated path, the path never crosses walls, and movement still works.

Result / AI Work Summary

Implemented BFS shortest-path highlighting and H toggle behavior. The implementation respects the maze's actual openings and recalculates after maze regeneration. Committed separately as: Add shortest path hint.

4. Task 2 — Fog of War

User/Vibe Coding Prompt

Implement ONLY Task 2 from README.md: Fog of War.

Reveal only maze cells within a radius of 3 cells around the player. Use a transparent pygame SRCALPHA surface for the dark overlay. The visible area must follow the player. Preserve BFS, wall collision, R, and all existing functionality. Do not implement Tasks 3–4.

Test distant cells, nearby visibility, player movement, H, R, and wall collision.

Result / AI Work Summary

Implemented a dark SRCALPHA fog overlay with a 3-cell visibility radius that follows the player. Existing BFS, movement, collision, and regeneration were preserved. Committed separately as: Add fog of war.

5. Task 3 — Timer Leaderboard

User/Vibe Coding Prompt

Implement ONLY Task 3 from README.md: Timer Leaderboard.

Start timing when a maze run begins and stop when the player reaches the exit. Persist the best five completion times in leaderboard.json, sorted fastest-first. Create the file if missing and handle invalid JSON safely. Display the leaderboard on the win screen. R starts a fresh run and resets the timer. Preserve Tasks 1–2 and wall collision. Do not implement Task 4.

Test timing, JSON creation, sorting, five-entry limit, persistence after restart, R reset, BFS, and fog.

Result / AI Work Summary

Implemented run timing and persistent top-five leaderboard storage in leaderboard.json, including safe handling of missing/invalid data. The timer resets for new runs and the leaderboard is displayed after completion. Committed separately as: Add timer leaderboard.

6. Task 4 — Difficulty Tiers

User/Vibe Coding Prompt

Implement ONLY Task 4 from README.md: Larger Maze Difficulty Tiers.

Add a difficulty selection screen with exactly:

- Easy: 10x8
- Medium: 15x13
- Hard: 20x18

The selected difficulty must remain active. R regenerates the same difficulty. Preserve timer, leaderboard, BFS, fog, wall collision, and controls. Do not remove existing functionality or implement unrelated changes.

Test all three dimensions and all previous features.

Result / AI Work Summary

Implemented the difficulty selection screen and the three required maze sizes. The selected dimensions are used for maze generation, R preserves the selected difficulty, and all previous features remain functional. Committed separately as: Add difficulty tiers.

7. Final verification

User/Vibe Coding Prompt

Perform a final integration check against every README requirement. Verify original controls, BFS hint, fog of war, timer leaderboard, JSON persistence, difficulty selection, R regeneration, and compatibility between all features. Identify any regressions before final submission.

Result / AI Work Summary

Final testing confirmed that Easy, Medium, and Hard all work and that the previously implemented BFS hint, fog of war, wall collision, timer, leaderboard, and R regeneration continue to work together.

Git Commit Evidence

Order	Commit
1	Fix broken maze wall collision
2	Add shortest path hint
3	Add fog of war
4	Add timer leaderboard
5	Add difficulty tiers

Final status: All four README tasks were implemented individually and tested. The project was developed through an iterative Vibe Coding workflow with separate commits for the required tasks.

Note: This is a formatted record of the Vibe Coding interaction and implementation workflow. If the instructor specifically requires the platform's native exported ChatGPT conversation file, retain/export the original conversation as well.